IOT - 2

PROJEKAT 2

Veljko Veljović 1937

ELEKTRONSKI FAKULTET NIŠ

Arduino

```
void setup()
{
    Serial.begin(115200);

    pinMode(LEDR, OUTPUT);
    pinMode(LEDB, OUTPUT);
    pinMode(LEDG, OUTPUT);
    digitalWrite(LEDR, HIGH);
    digitalWrite(LEDB, HIGH);
    digitalWrite(LEDG, HIGH);

    if (!IMU.begin()) {
    Serial.println("Failed to initialize IMU!");
    while (1);
}
```

U okviru drugog projekta korišćen je akcelerometar kako bi se odredila pozicija uređaja. Na osnovu očitanih podataka utvrđuje se da li se uređaj nalazi u horizontalnom, vertikalnom ili dijagonalnom položaju.

Arduino

Prepoznavanje komandi

```
for (size_t ix = 0; ix < EI_CLASSIFIER_LABEL_COUNT; ix++) {
    if (result.classification[ix].value > 0.8) {
        if (strcmp(result.classification[ix].label, "start") == 0) {
            reading = true;
            digitalWrite(LEDG, LOW);
            digitalWrite(LEDR, HIGH);
            Serial.println("Start");
        }
        else if (strcmp(result.classification[ix].label, "finish") == 0) {
            reading = false;
            digitalWrite(LEDG, HIGH);
            digitalWrite(LEDR, LOW);
            digitalWrite(LEDB, HIGH);
            Serial.println("Finish");
        }
    }
```

Određivanje pozicije

```
if (reading) {
    float x, y, z;

    if (IMU.accelerationAvailable()) {
        IMU.readAcceleration(x, y, z);

        String orientation;
        float absZ = abs(z);
        float absX = abs(x);
        float absY = abs(y);

        if (absZ > 0.7 && absX < 0.5 && absY < 0.5) {
            orientation = "HORIZONTAL";
        }
        else if ((absX > 0.7 || absY > 0.7) && absZ < 0.5) {
            orientation = "VERTICAL";
        }
        else {
            orientation = "DIAGONAL";
        }

        Serial.print("X: ");
        Serial.print(x);
        Serial.print("  Y: ");
        Serial.print(y);
        Serial.print("  Z: ");
        Serial.print(z);
        Serial.print(" Label: ");
        Serial.println(orientation);

        if (x < 0 || y < 0 || z < 0) {
            digitalWrite(LEDB, LOW);
        }
        else {
            digitalWrite(LEDB, HIGH);
        }
    }
}
```

Proxy.py

```python
import serial
import paho.mqtt.client as mqtt

ser = serial.Serial('/dev/ttyACM0', 115200, timeout=1)

client = mqtt.Client()
client.connect("localhost",1883,60)

while True:
    line = ser.readline().decode(errors='ignore').strip()
    if line:
        print("Primljeno ", line)
        client.publish("sensors/nano33", line)
```

consumer.py

```python
import paho.mqtt.client as paho
from influxdb import InfluxDBClient
from datetime import datetime
import numpy as np
import tensorflow as tf
import pickle

model = tf.keras.models.load_model('imu_orientation_model.h5')
with open('label_encoder.pkl', 'rb') as f:
    label_encoder = pickle.load(f)

def predict_orientation(x, y, z):
    sample = np.array([[x, y, z]])
    pred = model.predict(sample, verbose=0)
    return label_encoder.classes_[np.argmax(pred)]

BATCH_SIZE = 10
```

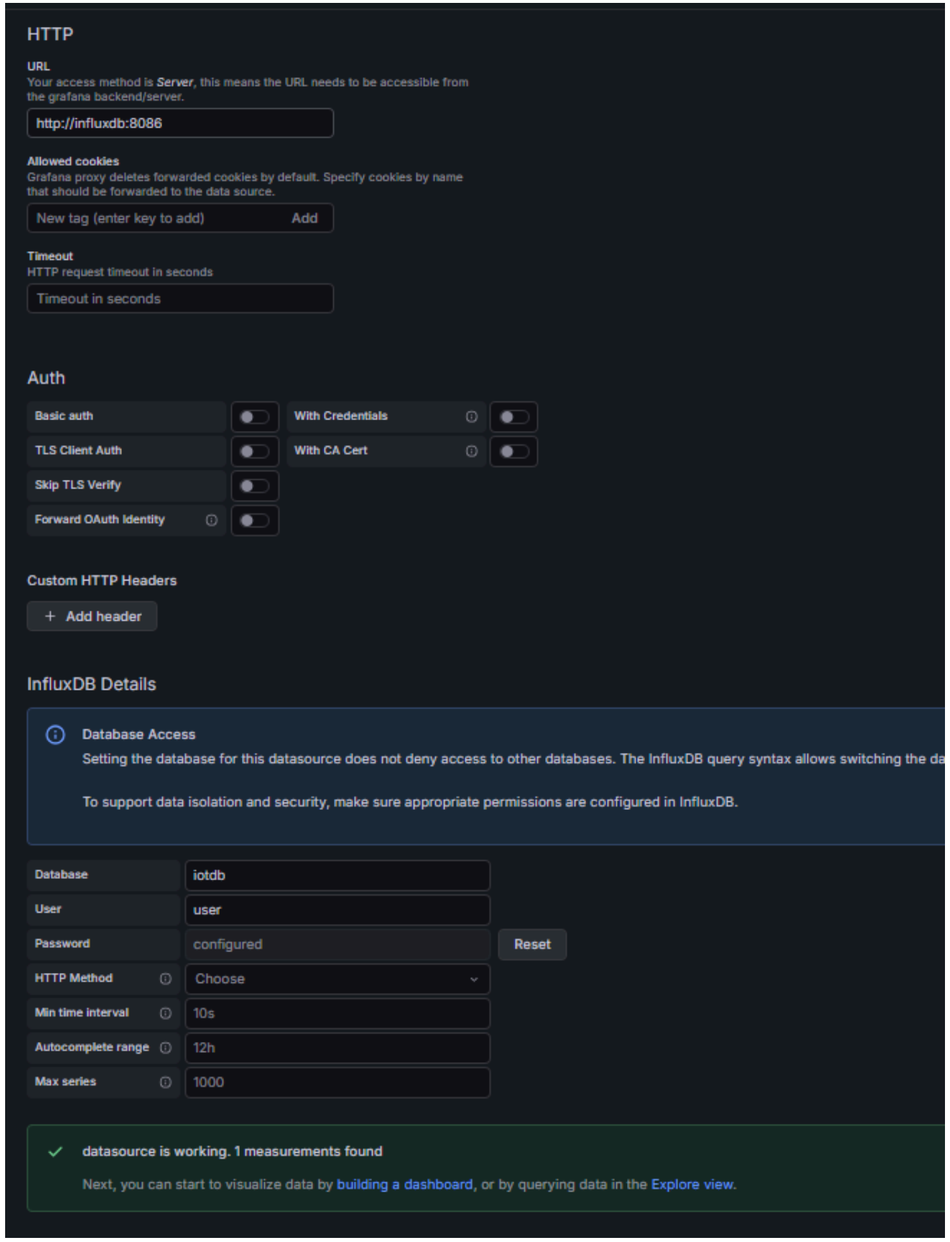
```python
def add_to_buffer(x,y,z,orientation):
    data = {
        "measurement" : "imu_data",
        "tags" : {"orientation" : orientation},
        "fields" : {
        "x": x,
        "y": y,
        "z": z},
        "time" : datetime.utcnow().isoformat()
        }
    if orientation == "VERTICAL":
        vertical_array.append(data)
        if len(vertical_array) == BATCH_SIZE:
            influxClient.write_points(vertical_array)
            print("vertical buffer stored")
            vertical_array.clear()

    elif orientation == "HORIZONTAL":
        horizontal_array.append(data)
        if len(horizontal_array) == BATCH_SIZE:
            influxClient.write_points(horizontal_array)
            print("horizontal buffer stored")
            horizontal_array.clear()

    else:
        diagonal_array.append(data)
        if len(diagonal_array) == BATCH_SIZE:
            influxClient.write_points(diagonal_array)
            print("diagonal buffer stored")
            diagonal_array.clear()
```

consumer.py

```python
def on_connect(client,userdata,flags,rc):
    print("Connected with code:", rc)
    client.subscribe("sensors/nano33")

def on_message(client, userdata, msg):
    message = msg.payload.decode()
    print("Primljeno:", message)
    coords = {}
    if "X:" in message:
        parts = message.split()
        x = float(parts[1])
        y = float(parts[3])
        z = float(parts[5])
        label = predict_orientation(x, y, z)
        print(x)
        print(y)
        print(z)
        print(label)
        if label == "VERTICAL":
            print('alert')
        add_to_buffer(x,y,z,label)
```

```python
influxClient = InfluxDBClient(
    host = 'localhost',
    port = 8086,
    username='user',
    password = 'userpass',
    database='iotdb'
)
if {'name': 'iotdb'} not in influxClient.get_list_database():
    influxClient.create_database('iotdb')

horizontal_array = []
vertical_array = []
diagonal_array = []
client = paho.Client()
client.on_connect = on_connect
client.on_message = on_message

client.connect("localhost", 1883, 60)
client.loop_forever()
```

Grafana


HTTP
URL
Your access method is Server, this means the URL needs to be accessible from the grafana backend/server.
http://influxdb:8086
Allowed cookies
Grafana proxy deletes forwarded cookies by default. Specify cookies by name that should be forwarded to the data source.
New tag (enter key to add)
Add
Timeout
HTTP request timeout in seconds
Timeout in seconds
Auth
Basic auth
With Credentials
TLS Client Auth
With CA Cert
Skip TLS Verify
Forward OAuth Identity
Custom HTTP Headers
+ Add header
InfluxDB Details
Database Access
Setting the database for this datasource does not deny access to other databases. The InfluxDB query syntax allows switching the da
To support data isolation and security, make sure appropriate permissions are configured in InfluxDB.
Database
iotdb
User
user
Password
configured
Reset
HTTP Method
Choose
Min time interval
10s
Autocomplete range
12h
Max series
1000
datasource is working. 1 measurements found
Next, you can start to visualize data by building a dashboard, or by querying data in the Explore view.

trainer.py

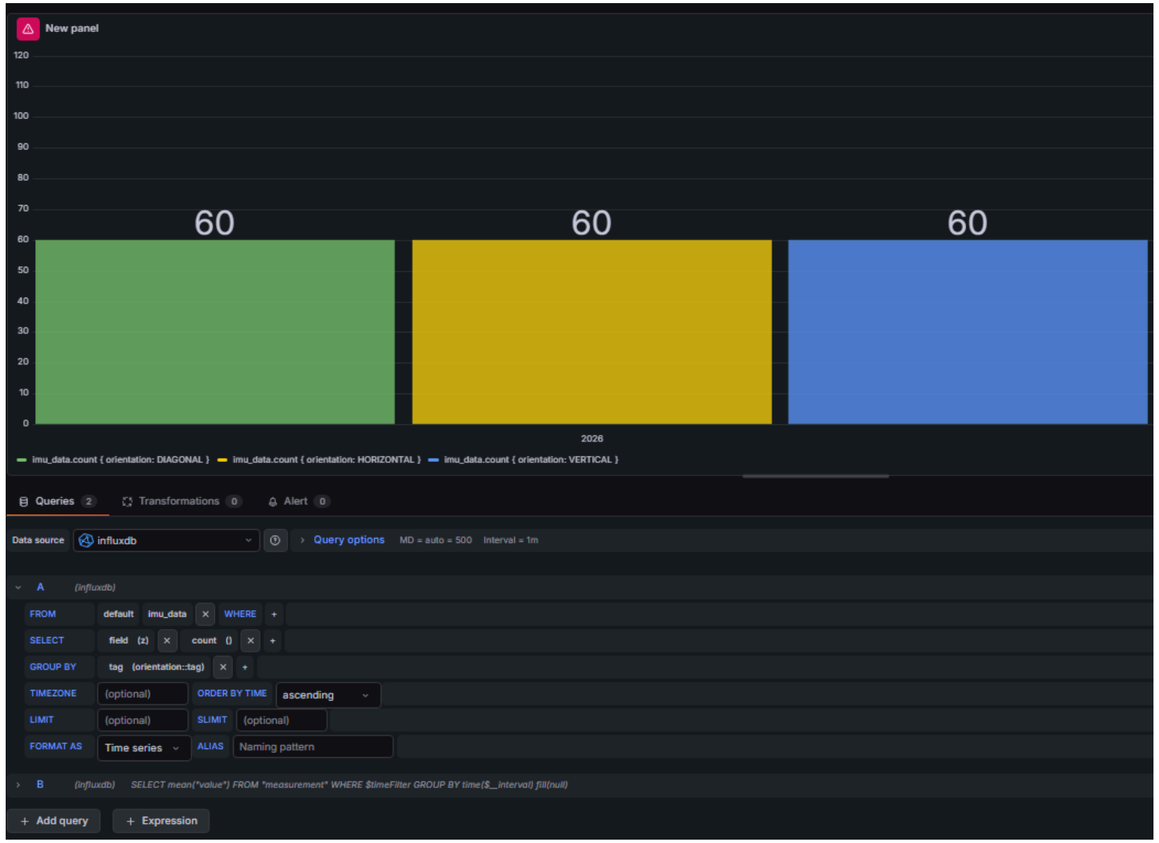
```python
from influxdb import InfluxDBClient
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import tensorflow as tf
from tensorflow import keras
import pickle
import matplotlib.pyplot as plt
import seaborn as sns

print('a')
client = InfluxDBClient(host='localhost', port = 8086, database='iotdb')
query = 'SELECT x, y, z, orientation FROM imu_data'
result = client.query(query)
print('b')
points = list(result.get_points())
df = pd.DataFrame(points)

print(f"Ukupno podataka: {len(df)}")
print(df.head())

df = df.dropna()

x = df[['x', 'y', 'z']].values
y = df['orientation'].values

print(x)
print(y)
```

```python
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

y_categorical = keras.utils.to_categorical(y_encoded)

x_train, x_test, y_train, y_test = train_test_split(
        x, y_categorical,
        test_size=0.2,
        random_state=42,
        stratify=y_encoded
    )
num_classes = len(label_encoder.classes_)

model = keras.Sequential([
        keras.layers.Input(shape=(3,)),
        keras.layers.Dense(16, activation='relu'),
        keras.layers.Dropout(0.2),
        keras.layers.Dense(8, activation='relu'),
        keras.layers.Dense(num_classes, activation='softmax')
    ])

model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
print(f"Train set: {len(x_train)}")
```

```python
history = model.fit(
        x_train, y_train,
        epochs=50,
        batch_size=32,
        validation_split=0.2,
        verbose=1
    )
print("\n" + "="*60)
print("EVALUACIJA")
print("="*60)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f} ({test_acc*100:.2f}%)")
print("\n" + "="*60)

model.save('imu_orientation_model.h5')
print("Model sačuvan: imu_orientation_model.h5")
with open('label_encoder.pkl', 'wb') as f:
    pickle.dump(label_encoder, f)
```

```python
print("\n" + "="*60)
print("TEST")
print("="*60)
def predict_orientation(x_val, y_val, z_val):
    sample = np.array([[x_val, y_val, z_val]])
    pred = model.predict(sample, verbose=0)
    pred_class = label_encoder.classes_[np.argmax(pred)]
    confidence = np.max(pred)
    print(f"X={x_val:.2f}, Y={y_val:.2f}, Z={z_val:.2f} → {pred_class} ({confidence:.2%})")
predict_orientation(0.02, -0.01, 0.98)   # horizontal
predict_orientation(0.95, 0.03, 0.05)    # vertical
predict_orientation(0.65, 0.10, 0.70)    # diagonal
```

Grafana


New panel
120
110
100
90
80
70
60
50
40
30
20
10
0
60
60
60
2026
imu_data.count { orientation: DIAGONAL }
imu_data.count { orientation: HORIZONTAL }
imu_data.count { orientation: VERTICAL }
Queries 2
Transformations 0
Alert 0
Data source
influxdb
Query options
MD = auto = 500
Interval = 1m
A
(influxdb)
FROM
default
imu_data
WHERE
SELECT
field (z)
count ()
GROUP BY
tag (orientation::tag)
TIMEZONE
(optional)
ORDER BY TIME
ascending
LIMIT
(optional)
SLIMIT
(optional)
FORMAT AS
Time series
ALIAS
Naming pattern
B
(influxdb)
SELECT mean("value") FROM "measurement" WHERE $timeFilter GROUP BY time($__interval) fill(null)
Add query
Expression

-0.0490
0.0157
0.0582
imu_data.mean { orientation: DIAGONAL }
imu_data.mean { orientation: HORIZONTAL }
imu_data.mean { orientation: VERTICAL }
Queries 2
Transformations 0
Data source
influxdb
Query options
MD = auto = 2218
Interval = 20s
Query inspector
A
(influxdb)
FROM
default
imu_data
WHERE
SELECT
field (x)
mean ()
GROUP BY
tag (orientation::tag)
TIMEZONE
(optional)
ORDER BY TIME
ascending
LIMIT
(optional)
SLIMIT
(optional)
FORMAT AS
Time series
ALIAS
Naming pattern

docker compose

```yaml
version: '3.1'
services:
  mosquitto:
    image: eclipse-mosquitto
    hostname: mosquitto
    container_name: mosquitto
    restart: unless-stopped
    ports:
      - "1883:1883"
      - "9001:9001"
    networks:
      - iot2
  influxdb:
    image: influxdb:1.8
    ports:
      - "8086:8086"
    environment:
      INFLUXDB_DB: iotdb
      INFLUXDB_ADMIN_USER: admin
      INFLUXDB_ADMIN_PASSWORD: admin12345
      INFLUXDB_USER: user
      INFLUXDB_USER_PASSWORD: userpass
    volumes:
      - ./influxdb-data:/var/lib/influxdb
    networks:
      - iot2
  grafana:
    image: grafana/grafana-enterprise:latest
    container_name: grafana
    restart: unless-stopped
    ports:
      - '3000:3000'
    networks:
      - iot2

networks:
  iot2:
    driver: bridge
```

Hvala na pažnji!